Automated Web Scanner for Secure Headers and CORS Flaws

Project Report

IBM SkillsBuild National Cybersecurity Bootcamp (BharatCares / CSRBOX)

Project Topic: Automated Web Scanner

Date: 22 October 2026

Submitted by: Soumadeep Mondal

1. Project Overview

This project delivers WebSecAudit, a Python command-line scanner that audits websites for missing security headers and CORS misconfiguration (Project Topic: Automated Web Scanner).

For each URL it is given, the tool sends a small number of read-only HTTP requests (GET and OPTIONS), compares the responses with recognised hardening practice, and produces a graded report. Every finding includes the evidence seen, a severity, a CWE identifier and remediation advice. The tool also checks cookie flags and the HTTP-to-HTTPS redirect, because they are closely tied to header hardening.

Objective. Automate the review a tester would otherwise do by hand with browser developer tools or curl, and make the result repeatable, scored and easy to share.

Scope. Passive configuration auditing only. The tool does not exploit vulnerabilities, send attack payloads, crawl sites or test application logic.

2. Background

Security headers and CORS rules are browser-enforced controls that limit what a page can do and which other sites can read its data; when they are missing or too permissive, attacks such as cross-site scripting (XSS), clickjacking and cross-site data theft become easier.

Header	What it protects against
Strict-Transport-Security (HSTS)	Forces HTTPS and blocks SSL-stripping downgrades
Content-Security-Policy (CSP)	Restricts script and resource sources, limiting the impact of XSS
X-Frame-Options / CSP frame-ancestors	Stops other sites framing the page (clickjacking)
X-Content-Type-Options	Stops browsers guessing (sniffing) content types
Referrer-Policy	Limits how much of a URL is leaked to other sites

Header	What it protects against
Permissions-Policy	Restricts browser features such as camera and geolocation

By default the same-origin policy stops one website reading another's responses. Cross-Origin Resource Sharing (CORS) lets a server relax this with the Access-Control-Allow-Origin header (ACAO). It becomes a flaw when the server reflects any Origin it receives, trusts the special origin "null", validates origins by substring or prefix, or combines these with Access-Control-Allow-Credentials: true. Any website can then make authenticated requests as the visitor and read the answers (CWE-942).

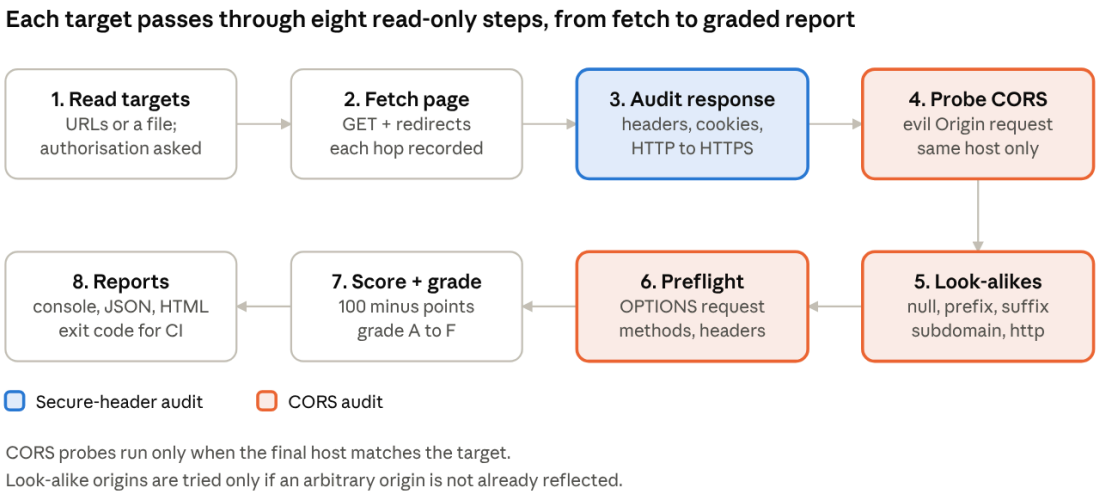
3. Tools and Technologies

The scanner needs only Python 3 and one third-party library.

- **Python 3** (developed and tested on 3.13).
- **requests**, the only external dependency, for HTTP calls.
- **Standard library:** argparse (command line), dataclasses, json and html (reports), ipaddress and urllib.parse (target handling), http.server (lab servers) and unittest (tests).
- **Local lab servers** written for this project, so the scanner can be demonstrated and tested without touching any third-party site.

4. Design and Methodology

The scanner is a fixed pipeline: it fetches the page once, audits that response, sends a few extra CORS probes, then scores and reports.



Scanner workflow: eight read-only steps from fetch to graded report

Steps 3 to 6 hold the audit logic. The header, cookie and transport checks are pure functions over the captured response, so they can be unit tested without a network. The CORS checks first send an untrusted Origin. If the server does not reflect it, they try look-alike origins to expose weak validation: the value null, an origin that starts with the real hostname, one that ends with it, an

invented subdomain and the http:// version of the site. A final OPTIONS preflight shows which methods and headers an untrusted origin is allowed to use.

5. Checks Implemented

The scanner runs four groups of checks, with the secure-header and CORS groups as the core of the project.

Group	Check	Severity
Headers	HSTS missing, invalid, disabled (max-age=0) or short	Medium to low
Headers	CSP missing, report-only, 'unsafe-inline', 'unsafe-eval', wildcard or scheme-wide sources, no object-src or base-uri	Medium to low
Headers	Clickjacking protection missing (X-Frame-Options or frame-ancestors), obsolete ALLOW-FROM	Medium to low
Headers	X-Content-Type-Options, Referrer-Policy (missing, weak, unsafe-url) and Permissions-Policy	Medium to low
Headers	Version disclosure (Server, X-Powered-By and similar); COOP/COEP/CORP; legacy X-XSS-Protection	Low to info
CORS	Arbitrary Origin reflected, with or without credentials	Critical (medium without credentials)
CORS	Origin "null" trusted	High with credentials, low without
CORS	Prefix or suffix origin-validation bypass (look-alike origins accepted)	High with credentials, medium without
CORS	Wildcard ACAO; any subdomain trusted; plain-HTTP origin trusted by an HTTPS site	Low to medium
CORS	Dynamic ACAO without Vary: Origin	Low
CORS	Preflight grants PUT/DELETE/PATCH, arbitrary headers, very long max-age or Private Network Access to an untrusted origin	Medium to info

Group	Check	Severity
Cookies	Missing Secure, HttpOnly or SameSite; SameSite=None without Secure; broken __Host- and __Secure- prefixes	Medium to low
Transport	Page served over plain HTTP; no or insecure HTTP-to-HTTPS redirect	High to medium

Cookie findings rank higher when the cookie name looks like a session or authentication cookie. Cookie values are never printed.

6. Scoring Model

Each target starts at 100 points and loses points for every finding, then receives a letter grade.

Severity	Points deducted per finding
Critical	40
High	25
Medium	12
Low	4
Info	0

Grades are A for 90 and above, B for 80 and above, C for 65 and above, D for 50 and above, and F below 50. Two overrides apply: any critical finding forces an F, and any high finding caps the grade at D, so one serious flaw cannot be hidden by many passed checks.

7. Test Environment

Four local HTTP servers on 127.0.0.1 (file `lab_servers.py`) were configured differently on purpose, so the scanner could be validated without scanning any third-party website.

Port	Name	Configuration
8081	vulnerable	No hardening headers, version banners (Apache/2.4.49, PHP/7.4.3), cookies without flags, weak CSP, reflects any Origin together with credentials, permissive preflight
8082	weakcors	Good security headers, but origin check is a substring match and "null" is trusted, with credentials

Port	Name	Configuration
8083	wildcard	Access-Control-Allow-Origin: * and only partial header coverage
8084	hardened	Full header set, a __Host- cookie with Secure, HttpOnly and SameSite=Strict, and an exact CORS allow-list

Because the lab runs over plain HTTP on localhost, the scanner deliberately skips HSTS, Secure-cookie and HTTP-to-HTTPS checks for loopback targets and records an informational note instead.

8. Results

The scanner graded the four lab servers exactly in line with how they were built: F, F, C and A.

Server	Critical	High	Medium	Low	Info	Score	Grade
:8081 vulnerable	1	0	6	12	4	0/100	F
:8082 weakcors	0	2	1	0	1	38/100	F
:8083 wildcard	0	0	1	3	2	76/100	C
:8084 hardened	0	0	0	0	1	100/100	A

Example finding (:8081, critical). The scanner sent a request with Origin: https://evil-attacker.example and the server answered with Access-Control-Allow-Origin: https://evil-attacker.example and Access-Control-Allow-Credentials: true. Any website could therefore make authenticated requests as a visitor and read the responses. Recommended fix: validate the Origin against an exact allow-list and never combine reflection with credentials.

Example finding (:8082, high). Look-alike origins http://localhost.evil-attacker.example and http://evillocalhost were both accepted, which shows substring matching instead of an exact comparison. The tool also detected that "null" was trusted with credentials.

The hardened server produced no finding above informational level, which confirms the checks do not raise false alarms on a correct configuration. The full console, JSON and HTML output of this run is included in the `sample_output` folder of the project.

9. Testing and Validation

A suite of 37 automated tests (file `test_scanner.py`, run with `python3 -m unittest`) passes in about 1.5 seconds and needs no internet access.

- **Rule tests** feed hand-built responses to every header, cookie and scoring rule, including edge cases such as a CSP that uses a nonce, a multi-value Referrer-Policy and case-insensitive header names.

- **Integration tests** scan all four lab servers end to end and assert the expected findings, severities, grades and that a hardened server stays clean.
- **CLI and report tests** cover the authorisation gate, target files, JSON and HTML output, the --fail-on exit code, and HTML escaping of attacker-controlled header values.

Testing also found a real mistake in the lab itself. Python's http.server adds its own Server banner, so the "hardened" server was first reported as disclosing a version. The scanner was right; the lab server was corrected to send a single Server header and the suite then passed.

10. Ethics and Responsible Use

The scanner must only be used on systems the user owns or has written permission to test, and the tool is built to make that hard to forget.

- It asks the user to confirm authorisation before sending anything; in non-interactive use it refuses to run unless --yes is passed (exit code 3).
- It is read-only: GET and OPTIONS requests only, no exploit payloads, no crawling and no fuzzing.
- It is polite: a 0.5 second delay between requests by default, roughly 3 to 12 requests per target, and a User-Agent that identifies the tool.
- If a target redirects to a different host, active CORS probing is skipped because the new host may be outside the authorised scope.
- Cookie values are never printed, and the HTML report escapes every dynamic value and carries its own restrictive CSP.

11. Limitations and Future Work

The scanner is a configuration auditor, not a full vulnerability scanner, and its results should be reviewed by a person.

Limitations

- It audits only the URL it is given; CORS behaviour can differ per path, so API endpoints should be passed in separately.
- CSP analysis is heuristic: it detects common weaknesses but does not fully evaluate the policy.
- It does not test application logic (XSS, SQL injection) or analyse TLS certificates and ciphers.
- The HTTPS-specific rules (HSTS, Secure cookies, HTTP-to-HTTPS redirect) are covered by unit tests but were validated only against the local lab, not against a live HTTPS site.

Future work

- Read a list of API endpoints from an OpenAPI file and scan each one.
- Add TLS certificate and protocol checks.
- Use a full CSP evaluator and compare scans over time to show regressions.
- Export SARIF so findings appear in code-hosting security dashboards and CI pipelines.

12. Conclusion

WebSecAudit meets the project topic: a Python script that audits sites for secure-header and CORS flaws. It turns a manual review into a repeatable, scored report, separates harmless findings from serious ones (such as credentialed origin reflection), and gives a fix for each. Against the four lab servers it graded the vulnerable ones F, F and C and the hardened one A, and its 37 automated tests pass. Its built-in authorisation prompt, read-only requests and rate limiting keep it suitable for responsible, authorised use.

13. How to Run and References

Install the one dependency and scan a target you are authorised to test.

```
        pip install -r requirements.txt
        python3 web_security_scanner.py https://your-site.example --yes
        python3 web_security_scanner.py -f targets.txt --json out.json --html out.html
--fail-on high --yes
```

To try it safely, start the lab with `python3 lab_servers.py`, scan http://localhost:8081/ to http://localhost:8084/, and run the tests with `python3 -m unittest -v`.

References

- MDN: Cross-Origin Resource Sharing (CORS)
- MDN: Content Security Policy
- MDN: Strict-Transport-Security
- OWASP HTTP Headers Cheat Sheet
- PortSwigger Web Security Academy: CORS